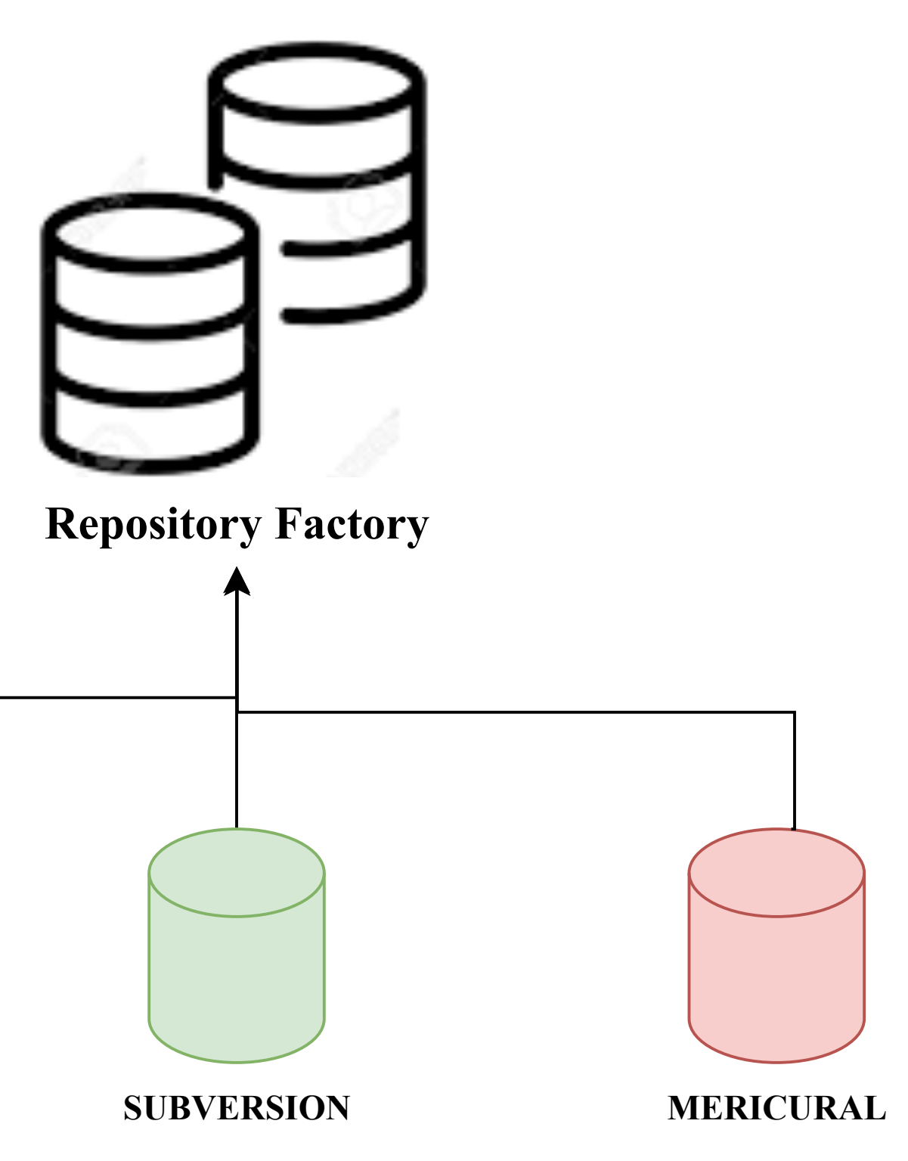
Factory Method Pattern

The Factory Method pattern provides a way to encapsulate object creation. Instead of instantiating objects directly using a constructor, this pattern delegates the instantiation process to a specialized method, often called a factory method. This pattern is particularly useful when there is a need to decouple object creation from its usage, allowing for more flexibility and easier maintenance.

Imagine a software development tool that must be able to work with several different kinds of source code repositories, including Git, Subversion, and Mercurial. Each repository type provides a unique method for activities such as commit, pull, and push. When additional repository types need to be added or when actions related to a particular repository change, implementing repository operations directly within the client code can result in a jumbled and difficult-to-maintain codebase.

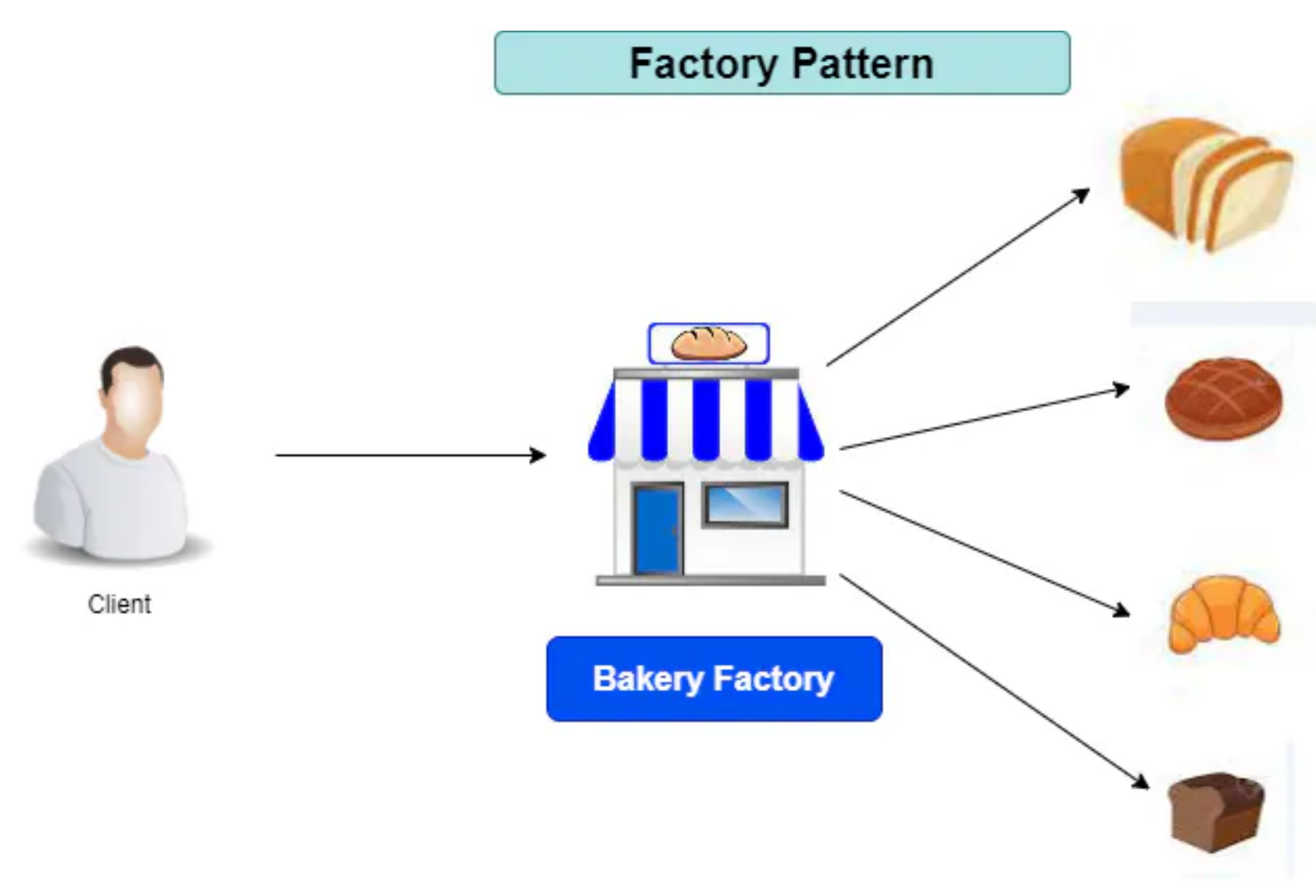
It is possible to use the Factory Method pattern to abstract the process of creating various repository types. The tool creates these objects using a factory method instead of directly instantiating repository instances. The factory method uses configuration or input to decide what kind of repository object to construct. The image below shows an illustration of this.



Factory Pattern -Example

For instance, the `createRepository` method of a `RepositoryFactory` class may return a Git, Subversion, or Mercurial repository object, depending on the arguments. Despite their internal variations, each of these objects would be a subclass of a shared Repository interface, ensuring that client code could interact with them using a uniform set of methods.

Real-Life Example



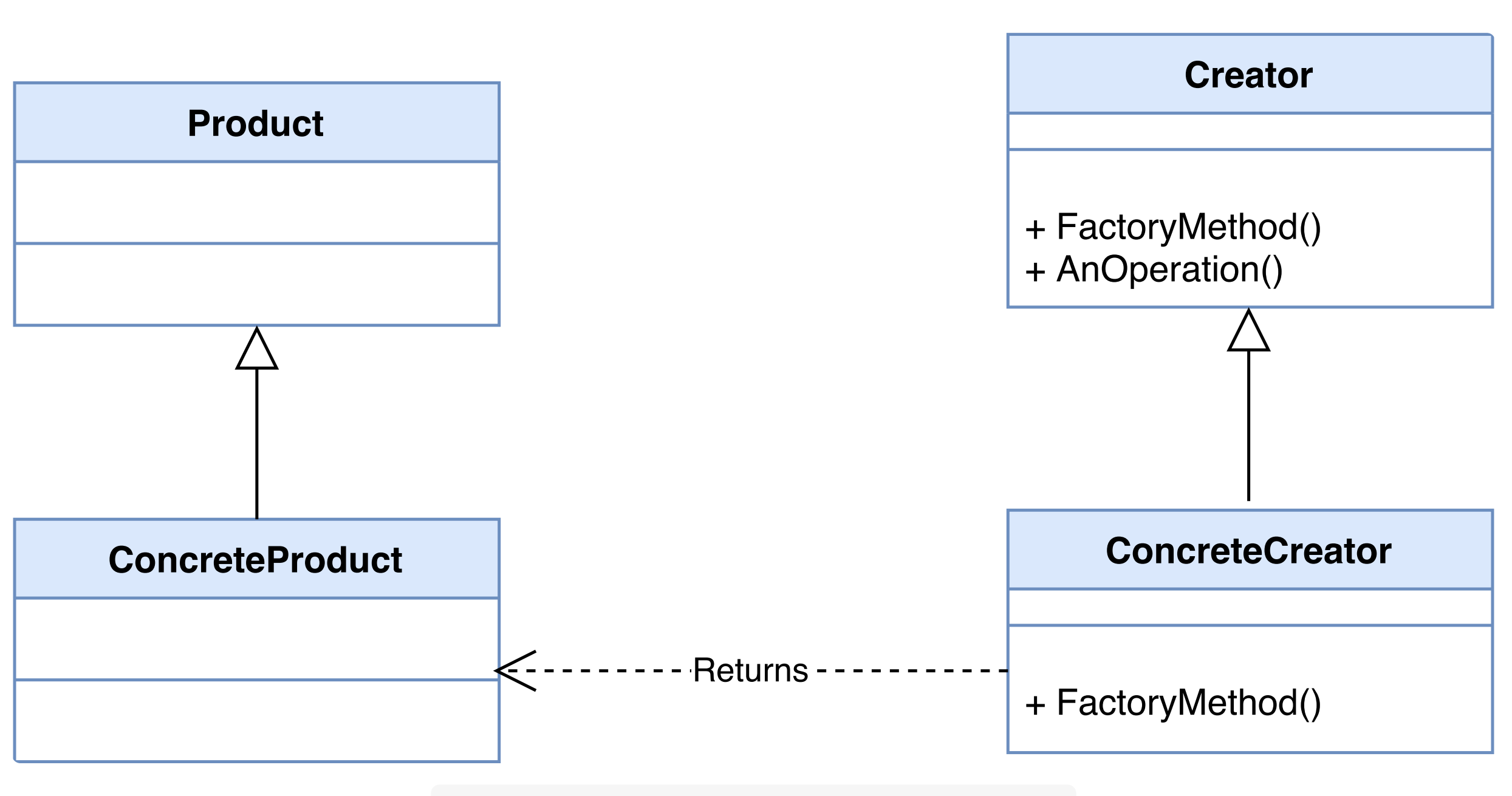
Factory Pattern -Example

Have you ever visited a bakery and requested for a particular kind of bread without being aware of how it is made? The Factory Pattern, in fact, does just that! You don't have to worry about how things were made when they are delivered to you. Let's suppose you're looking to buy a new computer. Your needs will determine the requirements for certain jobs. For instance, a graphic designer could need a powerful graphics card, whereas a writer might value a comfy keyboard more. But don't worry about assembling the parts yourself! When you go to the store, they "produce" the perfect computer for you based on your specifications. It's just like the Factory Pattern in programming! It creates things based on what you need without revealing the logic behind it.

Class Diagram of Factory Pattern

The Factory Method Pattern specifies an interface for creating an object while allowing subclasses to change the type of objects created. Typically, the class diagram has the following elements:

1. **Product:** This interface or abstract class defines the objects the factory method will create.
2. **ConcreteProduct:** These are the classes that implement the `Product` interface or extend the abstract class, defining objects to be created by the concrete factory.
3. **Creator:** This abstract class declares the factory method, which returns an object of type `Product`. The `Creator` might also provide an implementation of the factory method as a default.
4. **ConcreteCreator:** These are classes that implement or override the factory method to return an instance of `ConcreteProduct`.



Factory Method Pattern - Class Diagram

How It Works

- Instead of calling a constructor directly to instantiate an object, the client calls the factory method on the Creator (or the Concrete Creator).
- This method typically contains the logic to determine which Concrete Product to instantiate based on some parameters or runtime logic.
- The client uses the product interface to interact with the object, allowing the client code to work with different types of products without being tightly coupled to their concrete classes.

Implementation of Factory Pattern

For our real-world example, let's consider a simple scenario: A `ShapeFactory` that produces different types of shapes.

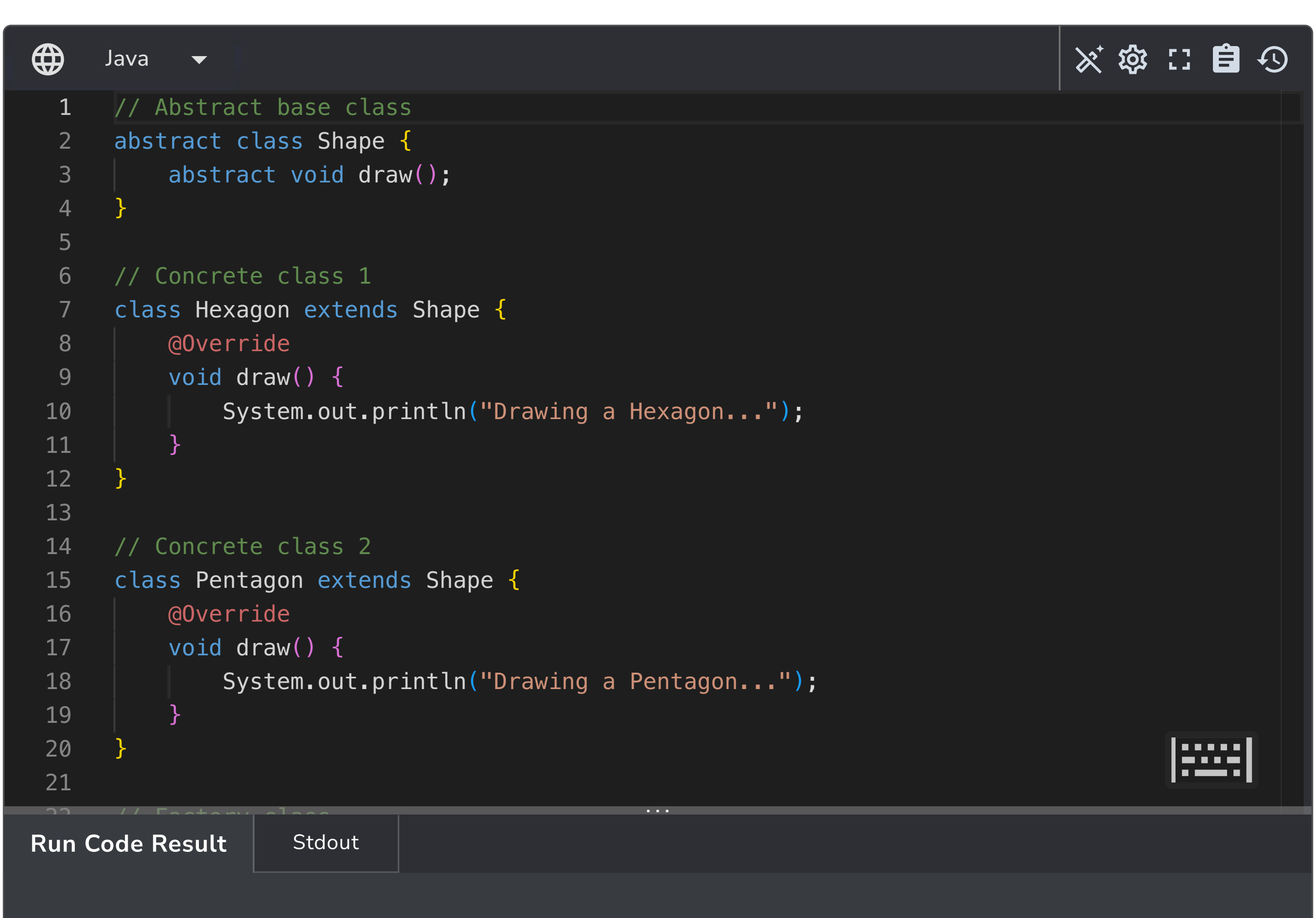
We will look at the pseudocode and implementation of `ShapeFactory` in different programming languages.

Pseudocode

```
CLASS Shape
   METHOD draw()  // abstract

CLASS Hexagon INHERITS Shape
   METHOD draw()
      PRINT "Drawing a Hexagon ... "

CLASS Pentagon INHERITS Shape
   METHOD draw()
      PRINT "Drawing a Pentagon ... "

CLASS ShapeFactory
   METHOD getShape(type: STRING) RETURNS Shape
      IF type EQUALS "Hexagon"
         RETURN NEW Hexagon
      ELSE IF type EQUALS "Pentagon"
         RETURN NEW Pentagon
      ELSE
         RETURN NULL

// Client Code
FUNCTION main()
   hexagon = ShapeFactory.getShape("Hexagon")
   hexagon.draw()

   pentagon = ShapeFactory.getShape("Pentagon")
   pentagon.draw()
```

- To begin, we establish the primary `Shape` interface which includes a `draw` method that will be implemented by concrete classes.
- Following that, we have two concrete classes, `Pentagon` and `Hexagon`, which both implement the `Shape` interface. Each class has its own unique version of the `draw` function.
- Then, the `ShapeFactory` class is defined with a `getShape` function that takes a `Shape` type parameter. This method creates and returns a new instance of the appropriate shape based on the provided type. If the type does not match any known shapes, it returns `null`.
- Finally, we demonstrate how to use the `ShapeFactory` in the `main` function to create a circle and rectangle. To do so, we first create a new instance of the factory and then call its draw method.

Let's explore how to implement this example in various programming languages.



```java
// Abstract base class
abstract class Shape {
    abstract void draw();
}

// Concrete class 1
class Hexagon extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a Hexagon...");
    }
}

// Concrete class 2
class Pentagon extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a Pentagon...");
    }
}

```

You must run your code first

Applications of Factory Pattern

GUI Libraries:

Different platforms or operating systems may render GUI elements differently. By using the Factory Pattern, GUI components like buttons, windows, and menus can be created, ensuring that the right type of object is instantiated on various platforms without disclosing the creation mechanism to the client.

Database Connections:

Applications may have to control connections to many kinds of databases. Based on the database system type (such as MySQL, PostgreSQL, or MongoDB) provided, Factory Pattern can construct and deliver a suitable database connection object, hiding the instantiation details and giving the client a consistent interface.

E-Commerce Payments Processing

E-commerce platforms may allow various payment methods (such as PayPal, Stripe, and Credit Cards). Based on the chosen payment method, a Payment Gateway Factory might provide the appropriate payment processing object, ensuring that the client is isolated from the instantiation logic and can operate with a consistent interface for payment processing.

Pros and Cons

Pros	Cons
Flexibility in Object Creation: Provides flexibility in object creation by hiding the creation logic behind an interface or method.	**Complexity:** May increase code complexity, particularly if several factory methods or classes are added.
Decoupling: This technique reduces dependencies by separating the product's implementation from the client using it.	**Number of Classes:** Since every kind of product needs its own factory, there might be more classes and interfaces overall.
Single Responsibility Principle: Adheres to this principle by isolating the creation logic, making the system easier to maintain.	**Refactoring:** If factories aren't used in the original design, adding them later may necessitate extensive reworking.
Easy to Introduce New Products: Because client code is based on abstract interfaces, introducing new products has no impact on existing client code.	**Indirection:**Adds another level of abstraction, which can occasionally make debugging or understanding the program's flow more difficult.
Consistent Product Interface: All products created by the factory conform to a common interface, ensuring consistency.	**Initial Overhead:** Can be overkill for simpler applications where a factory method might not be necessary.

Summary

The Factory Method Pattern is a way to achieve loose coupling and greater flexibility in object creation. It's especially useful in systems where the type of objects to create can change depending on circumstances or where there are many types of objects that share a common base class or interface. However, this method may be too much for small projects and might occasionally complicate your code, particularly if you have many separate objects. It's really beneficial when your software needs to manage a large number of similar items.

← Previous
Singleton Pattern
Next →
Abstract Factory Pattern
☐ Mark as Completed